



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

Department of Computer Science and Engineering

2026-2027

Odd Semester

DATA STRUCTURES AND ALGORITHMS-3

(25CS2103E)

FLOWLAB: EDMONDS-KARP VISUALIZER

P Yasasri

2520030373

M Sri Nithya

2520030447

COURSE INSTRUCTOR

Dr. J Sirisha Devi

Professor

Department of Computer Science and Engineering

EDMONDS- KARP VISUALIZER

Case Study and Project Description

1. Introduction

FLOWLAB is an interactive web-based visualization system for the Edmonds-Karp maximum-flow algorithm. The project is designed to make maximum-flow computation easier to understand by combining algorithmic execution with a visual representation of the graph, capacities, augmenting paths, and residual capacities.

2. Problem Statement

Maximum-flow algorithms are often taught using numerical examples, tables, or program output. Although these approaches can provide the final answer, they can make it difficult to understand how each augmenting path is selected, how the bottleneck is determined, how the residual graph changes, and why the algorithm eventually stops. FLOWLAB addresses this by providing an interactive environment in which a user can construct a directed capacitated graph and observe the maximum-flow computation.

3. Objectives

The main objectives are:

- To provide an interactive interface for creating a directed capacitated graph.
- To allow selection of a source and destination (sink).
- To compute maximum flow using Edmonds-Karp.
- To demonstrate BFS-based augmenting-path selection.
- To identify the bottleneck capacity of each selected path.
- To represent residual capacities and reverse edges.
- To present the final maximum-flow value clearly.
- To improve conceptual understanding through visualization.

4. Case Study / System Description

The system represents a general flow network. Each node represents a vertex and each directed edge has a capacity. The user enters nodes, connects them using directed edges, assigns capacities, and selects a source and destination. After the graph is prepared, Edmonds-Karp is executed. BFS searches the residual graph for an augmenting path from source to destination. The minimum residual capacity on that path is the bottleneck, and that amount is added to the current flow. Residual capacities are then updated before the next BFS iteration.

5. Example

Consider a network with source S, intermediate vertices A and B, and destination T. Suppose $S \rightarrow A = 10$, $S \rightarrow B = 8$, $A \rightarrow B = 5$, $A \rightarrow T = 7$, and $B \rightarrow T = 10$. BFS finds an available path from S to T. For each selected path, the bottleneck is the smallest residual capacity on that path. The flow is increased by the bottleneck, the forward residual capacity is reduced, and the reverse residual capacity is increased. BFS is repeated on the updated residual graph until T cannot be reached through any positive-residual-capacity path. The accumulated flow is the maximum flow.

6. User Interaction and Visualization

The interface contains a graph-input area and a visualization area. Users can add nodes and directed edges, enter capacities, and choose the source and destination. Arrows indicate edge direction and labels show capacities. The source and destination are visually distinguished. During execution, the visualization can represent the selected augmenting path, residual edges, and capacity changes. The final result displays the maximum flow, source, destination, and number of augmenting paths found.

7. Significance of the Project

The project focuses on understanding the algorithm rather than only producing a numerical answer. Edmonds-Karp is an important maximum-flow technique, and maximum-flow models are applicable to network and resource-allocation problems. By making the internal steps visible, FLOWLAB can be used as a learning aid and as a small experimental environment for testing different flow-network examples.

8. Feasibility

The project is feasible because it uses standard web technologies and a well-defined polynomial-time algorithm. The frontend represents graphs interactively, while the algorithm operates on in-memory graph and residual-graph structures. The system does not require specialized hardware or external services for its core operation.

ALGORITHM / PSEUDO CODE

Step-by-step representation of the Edmonds-Karp algorithm and pseudo code.

1. Maximum-Flow Concept

A flow network is a directed graph in which every directed edge has a capacity. A source is the starting vertex and a sink (destination) is the ending vertex. The objective is to send as much flow as possible from the source to the sink without exceeding edge capacities. For intermediate vertices, the amount entering and leaving must satisfy flow conservation.

2. Edmonds-Karp Algorithm

Edmonds-Karp is a specific implementation of the Ford-Fulkerson method. Its key difference is that BFS is used to select the augmenting path in the residual graph. Thus, BFS selects an augmenting path with the minimum number of edges among currently reachable paths. After a path is found, its bottleneck residual capacity is determined, the flow is augmented, and the residual graph is updated.

3. Step-by-Step Procedure

- Step 1: Create the original directed graph from the user-provided nodes and edges.
- Step 2: Initialize the residual graph with the original capacities and zero-capacity reverse edges where required.
- Step 3: Set maximum flow to zero.
- Step 4: Run BFS from the source over edges with positive residual capacity.
- Step 5: Store the parent of each visited vertex so the path can be reconstructed.
- Step 6: If the sink is not reached, terminate because no augmenting path exists.
- Step 7: Reconstruct the path from sink back to source using parent information.
- Step 8: Find the bottleneck as the minimum residual capacity on the path.
- Step 9: Subtract the bottleneck from every forward residual edge.
- Step 10: Add the bottleneck to every corresponding reverse residual edge.
- Step 11: Add the bottleneck to the total maximum flow.
- Step 12: Store the iteration information and repeat BFS on the updated residual graph.

4. BFS Pseudocode

BFS(source):

- create parent for every vertex and set it to null
- mark every vertex as unvisited
- enqueue source and mark source visited

while queue is not empty:

- current = dequeue()
- for every vertex next:
 - if next is unvisited and residual[current][next] > 0:

```

    parent[next] = current
    mark next visited
    enqueue next
    if next is sink:
        return parent

```

```

return null

```

5. Edmonds-Karp Pseudocode

```

EDMONDS_KARP(G, source, sink):
    construct residual graph R from G
    maxFlow = 0

```

```

    while an augmenting path P exists from source to sink in R using BFS:
        bottleneck = minimum residual capacity on P

```

```

        for every edge (u, v) in P:
            R[u][v] = R[u][v] - bottleneck
            R[v][u] = R[v][u] + bottleneck

```

```

        maxFlow = maxFlow + bottleneck
        record P, bottleneck, residual graph and maxFlow

```

```

    return maxFlow

```

6. Residual Graph

The residual graph allows the algorithm to reconsider previous decisions. A forward residual edge stores how much additional flow can still be sent through an original edge. A reverse residual edge stores how much previously sent flow can be cancelled or redirected. After every augmentation, forward residual capacity decreases and the corresponding reverse residual capacity increases.

7. Bottleneck Calculation

For an augmenting path containing several edges, the bottleneck is the smallest residual capacity among those edges. This value determines how much additional flow can safely be pushed along the entire path. For example, if a path has residual capacities 8, 5, and 10, the bottleneck is 5, so only 5 units can be added in that iteration.

8. Termination and Complexity

The algorithm stops when BFS can no longer reach the sink through a positive-residual-capacity edge. At that point there is no augmenting path and the current flow is the maximum flow. The Edmonds-Karp algorithm has time complexity $O(VE^2)$ for V vertices and E edges.

9. FLOWLAB Implementation Flow

In FLOWLAB, the user's graph input is converted into graph and residual-graph structures. BFS searches the residual graph, the selected path is reconstructed, and the bottleneck is computed. After residual capacities are updated, each iteration is stored in a steps array containing the path, bottleneck, flow added, cumulative maximum flow, and residual graph. This stored information supports the project's visualization.

CODE

```
// algorithm/edmondsKarp.js

function edmondsKarp(nodes, edges, source, sink) {
  const graph = {};
  const residualGraph = {};

  for (const node of nodes) {
    graph[node] = {};
    residualGraph[node] = {};
  }

  for (const edge of edges) {
    const { from, to, capacity } = edge;
    graph[from][to] = capacity;

    if (residualGraph[from][to] === undefined)
      residualGraph[from][to] = 0;

    if (residualGraph[to][from] === undefined)
      residualGraph[to][from] = 0;

    residualGraph[from][to] += capacity;
  }

  let maxFlow = 0;
  const steps = [];

  function bfs() {
    const parent = {};
    const visited = {};
    const queue = [];

    for (const node of nodes) {
      visited[node] = false;
      parent[node] = null;
    }

    queue.push(source);
    visited[source] = true;

    while (queue.length > 0) {
      const current = queue.shift();

      for (const next of nodes) {
        if (
          !visited[next] &&
          residualGraph[current] &&
          residualGraph[current][next] !== undefined &&
          residualGraph[current][next] > 0
        ) {
          parent[next] = current;
          queue.push(next);
          visited[next] = true;
        }
      }
    }
  }

  while (bfs()) {
    let flow = 0;
    let current = sink;
    while (current !== source) {
      const next = parent[current];
      flow = Math.min(flow, residualGraph[next][current]);
      residualGraph[next][current] -= flow;
      residualGraph[current][next] += flow;
      current = next;
    }
    maxFlow += flow;
    steps.push(flow);
  }

  return maxFlow;
}
```

```

    ) {
        parent[next] = current;
        visited[next] = true;
        queue.push(next);

        if (next === sink)
            return parent;
    }
}

return null;
}

while (true) {
    const parent = bfs();

    if (parent === null || !parent[sink])
        break;

    const path = [];
    let current = sink;

    while (current !== null) {
        path.push(current);
        current = parent[current];
    }

    path.reverse();

    let bottleneck = Infinity;

    for (let i = 0; i < path.length - 1; i++) {
        const from = path[i];
        const to = path[i + 1];

        bottleneck = Math.min(
            bottleneck,
            residualGraph[from][to]
        );
    }

    for (let i = 0; i < path.length - 1; i++) {
        const from = path[i];
        const to = path[i + 1];

        residualGraph[from][to] -= bottleneck;

        if (residualGraph[to][from] === undefined)
            residualGraph[to][from] = 0;
    }
}

```

```

    residualGraph[to][from] += bottleneck;
  }

  maxFlow += bottleneck;

  const residualGraphArray = [];

  for (const from of nodes) {
    if (residualGraph[from]) {
      for (const to of nodes) {
        if (residualGraph[from][to] !== undefined) {
          residualGraphArray.push({
            from: from,
            to: to,
            capacity: residualGraph[from][to]
          });
        }
      }
    }
  }

  steps.push({
    iteration: steps.length + 1,
    path: path,
    bottleneck: bottleneck,
    flowAdded: bottleneck,
    maxFlow: maxFlow,
    residualGraph: residualGraphArray
  });
}

return {
  source: source,
  sink: sink,
  maxFlow: maxFlow,
  steps: steps
};
}

module.exports = { edmondsKarp };

```

RESULTS

Results and Screenshots

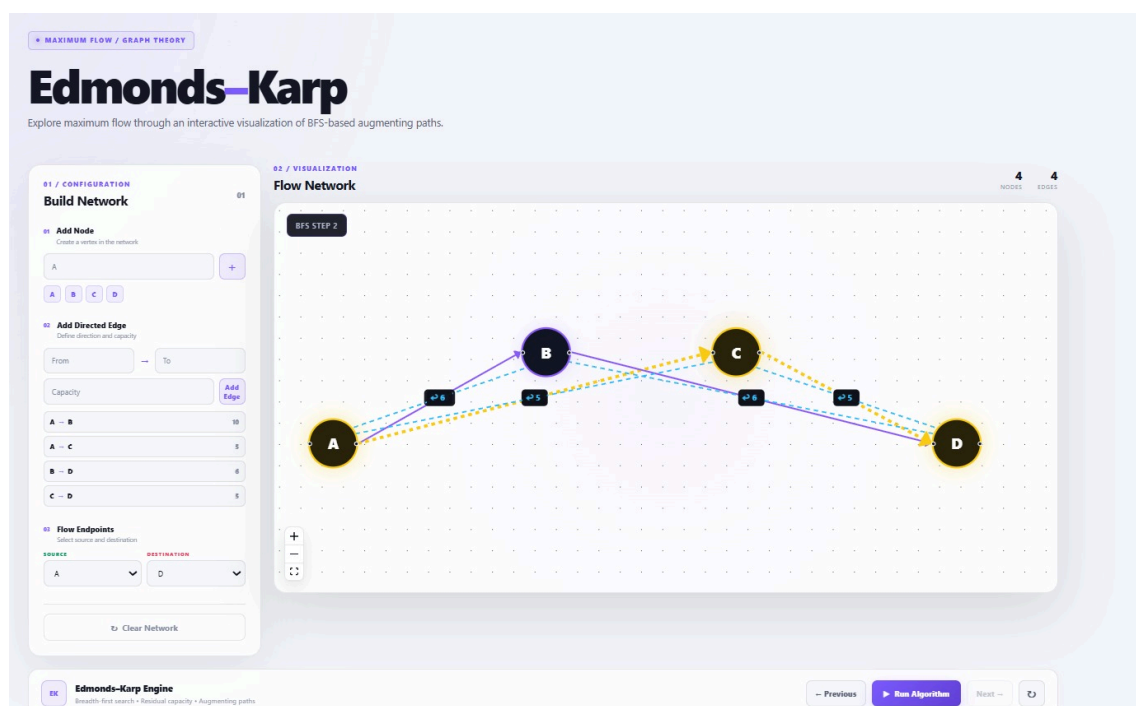
1. Expected Result

After valid nodes, directed edges, capacities, source, and destination are provided, the system executes Edmonds-Karp and displays the maximum-flow result. The output also identifies the selected source and destination and records the number of augmenting paths found.

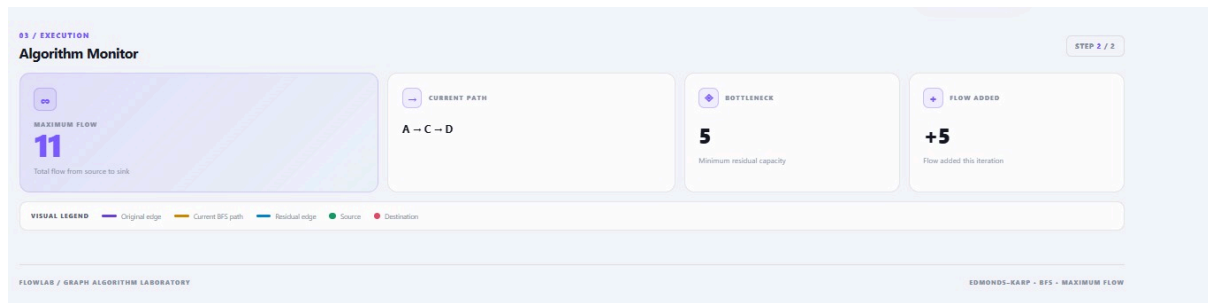
2. Visualization Result

The graph visualization represents the source and destination using distinct node styling. Original directed edges display their capacities. During an algorithm step, the selected augmenting path can be highlighted and residual edges can be shown using a different visual treatment. This allows the user to observe how the residual network changes after each augmentation.

3. Screenshot 1 – Graph Input & Visualization



5. Screenshot 2 – Algorithm Result



6. Conclusion

FLOWLAB provides an interactive way to construct a directed capacitated network and compute its maximum flow using the Edmonds-Karp algorithm. The project emphasizes the internal working of the algorithm by representing BFS-based augmenting paths, bottleneck capacities, residual updates, and the final maximum flow. It therefore serves as a practical visualization and learning tool for maximum-flow concepts.